

Bilkul. Ab main tumhe **AI Document Intelligence SaaS (RAG + Automation + MERN)** ka roadmap itna seedha dunga ki tum bina confused hue start kar sako.

Main isko aise bana raha hoon:

kya banana hai → kis order me banana hai → har step me kya karna hai → kab next step pe jana hai

Ye project tumhare liye **resume-level + internship-level + real-world level** hoga.

Project ka final idea

Tum ek web app banaoge jahan:

- user signup/login karega
- PDF upload karega
- system PDF ka text nikalega
- text ko chunks me divide karega
- chunks ko embeddings me convert karega
- vector DB me store karega
- user question puchega
- AI relevant chunks laake answer dega
- answer source ke saath show hoga
- extra features: summary, key points, quiz, flashcards, chat history

Ye hi hota hai:

AI Document Intelligence SaaS

Final Tech Stack

Frontend

- React
- Tailwind CSS
- Axios
- React Router
- Zustand ya Redux Toolkit
- React Hook Form
- Shadcn UI optional

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose

- JWT Auth
- Multer (file upload)
- pdf-parse ya pdfjs
- LangChain
- Gemini API ya OpenAI API
- Pinecone / MongoDB Atlas Vector Search / Qdrant

Recommended for you

Tumhare level ke hisab se:

- **Frontend:** React + Tailwind
- **Backend:** Node + Express + MongoDB
- **AI:** Gemini
- **Vector DB:** Pinecone ya MongoDB Atlas Vector Search

Agar easy start chahiye:

- embeddings + vector storage ke liye **Pinecone**
Agar full stack strong dikhana hai:
- **MongoDB Atlas Vector Search**

Project modules

Project ko 8 parts me tod do:

1. Project setup
2. Authentication
3. File upload
4. PDF text extraction
5. Chunking + embeddings + vector storage
6. RAG chat system
7. AI automation features
8. Deployment + polish

Sabse pehle kya outcome hona chahiye?

Tumhara target ye nahi hona chahiye ki "poora app ek saath banau".

Tumhara target ye hona chahiye:

Phase-wise result

- Phase 1: UI ready
- Phase 2: Auth working
- Phase 3: PDF upload working
- Phase 4: PDF text extract working
- Phase 5: embeddings save working

- Phase 6: AI Q&A working
- Phase 7: summary/flashcard/quiz working
- Phase 8: deploy + resume ready

30-day roadmap

Main 30 din ka roadmap de raha hoon. Agar tum daily 3–5 ghante doge to achha ban jayega.

WEEK 1 — Foundation + Auth + UI

Day 1 — Project planning

Aaj coding kam, planning zyada.

Kya karo:

- project ka naam decide karo
Example:
 - DocMind AI
 - AskMyPDF AI
 - InsightDoc
 - SmartDoc AI
- notebook ya Notion me likho:
 - project goal
 - features
 - pages
 - database models

Features final karo:

- signup/login
- dashboard
- upload PDF
- list uploaded documents
- ask questions from document
- document summary
- key points
- flashcards
- quiz
- chat history
- delete document

Pages final karo:

- Home page
- Login page

- Signup page
- Dashboard
- Upload page
- Document detail page
- Chat page
- Summary page
- Profile page

Database models socho:

- User
- Document
- Chat
- Flashcard
- Quiz

Aaj ka end result:

Tumhe exactly pata hona chahiye ki kya banana hai.

Day 2 — Folder setup

Frontend

Create React app with Vite.

Frontend folders:

- src/components
- src/pages
- src/layouts
- src/hooks
- src/services
- src/utils
- src/context or store

Backend

Backend folders:

- config
- controllers
- routes
- middleware
- models
- utils

- services
- uploads

Install frontend packages

- react-router-dom
- axios
- react-hook-form
- lucide-react
- tailwindcss

Install backend packages

- express
- mongoose
- dotenv
- cors
- bcryptjs
- jsonwebtoken
- multer
- pdf-parse
- langchain
- @google/generative-ai
- pinecone client or mongo vector dependencies

Aaj ka end result:

Frontend aur backend dono run hone chahiye.

Day 3 — Basic frontend layout

Pages banao:

- Home
- Login
- Signup
- Dashboard

Home page me:

- hero section
- features section
- CTA
- navbar
- footer

Dashboard dummy UI:

- sidebar
- topbar
- upload card
- recent docs section
- AI tools section

Aaj ka end result:

App dekhne me product jaisa lagna chahiye.

Day 4 — Backend basic server + MongoDB connection

Kya karo:

- express server banao
- app.js ya server.js setup karo
- MongoDB connect karo
- error middleware banao
- test route banao `/api/test`

Aaj ka end result:

Backend successfully DB se connect ho.

Day 5 — User model + auth routes

User model fields:

- name
- email
- password
- avatar optional
- createdAt

Auth routes:

- register
- login
- get current user

Features:

- password hash with bcrypt
- JWT generate karo
- protected route middleware banao

Aaj ka end result:

Postman me signup/login working.

Day 6 — Frontend auth integration

Kya karo:

- login form backend se connect
- signup form backend se connect
- token localStorage ya cookie me save
- protected routes setup
- logout function

Aaj ka end result:

User UI se signup/login kar sake.

Day 7 — Dashboard basic data flow

Kya karo:

- user login ke baad dashboard pe redirect
- current user fetch karo
- sidebar navigation add karo
- dashboard me dummy recent docs show karo

Aaj ka end result:

Authentication flow complete.

WEEK 2 — PDF upload + processing pipeline

Day 8 — Document model design

Document model fields:

- userId
- title
- originalFileName
- fileUrl / localPath
- extractedText
- summary
- totalPages
- status
- createdAt

Optional:

- tags
- fileSize

Status values:

- uploaded
- processing
- completed
- failed

Aaj ka end result:

Document schema ready.

Day 9 — File upload API**Kya karo:**

- Multer setup karo
- PDF upload route banao
- sirf PDF allow karo
- file size limit rakho
- uploaded file ko local ya cloud pe save karo

Route:

- POST `/api/documents/upload`

Aaj ka end result:

PDF backend pe upload ho jaye.

Day 10 — Frontend upload page**Kya karo:**

- drag and drop upload UI
- file picker
- upload progress
- success / error messages

Important:

- user ko clean UI do
- upload hone ke baad document list page pe le jao

Aaj ka end result:

UI se PDF upload ho.

Day 11 — PDF text extraction

Kya karo:

- uploaded file read karo
- `pdf-parse` se text nikalo
- extracted text DB me save karo
- page count nikal sakte ho to save karo

Route logic:

upload → parse PDF → save extracted text

Check karo:

- text aa raha hai ya messy aa raha hai
- kuch PDF image-based honge, wo issue दे sakte hain
- start me text PDFs pe focus karo

Aaj ka end result:

PDF ka text database me store ho.

Day 12 — Document listing

Kya karo:

- user ke uploaded documents fetch karo
- dashboard pe list dikhao
- document card me:
 - title
 - upload date
 - status
 - open button
 - delete button

Routes:

- GET all user docs
- GET single doc
- DELETE doc

Aaj ka end result:

Uploaded PDF dashboard me dikhna chahiye.

Day 13 — Document detail page

Kya dikhao:

- document title
- upload date
- status
- extracted text preview
- buttons:
 - Ask AI
 - Generate Summary
 - Flashcards
 - Quiz

Aaj ka end result:

Single document page ready.

Day 14 — Clean up day

Aaj bugs fix karo.

Checklist:

- auth bug
- upload bug
- route protection
- bad file format handling
- empty states
- loading spinner

Aaj ka end result:

Week 2 stable hona chahiye.

WEEK 3 — RAG system build

Ye project ka heart hai.

Day 15 — Chunking samjho aur implement karo

PDF ka pura text AI ko direct nahi dena.

Kya karna hai:

- text ko small chunks me todna hai
- har chunk ka size around 500–1000 characters ya tokens
- overlap add karo

Example:

Chunk 1 = line 1–20

Chunk 2 = line 18–38

Ye overlap isliye hota hai taki context na tute.

Backend me utility banao:

```
chunkText(text)
```

Aaj ka end result:

Extracted text chunks me convert ho.

Day 16 — Embeddings generate karo

Kya karna hai:

- har chunk ko embeddings me convert karo
- embeddings matlab number vectors
- ye vector semantic meaning carry karta hai

Input:

chunk text

Output:

vector

Aaj ka end result:

Ek document ke chunks ke embeddings generate ho jaye.

Day 17 — Vector DB store

Kya karna hai:

- embeddings ko vector DB me save karo
- metadata ke saath save karo

Metadata example:

- userId
- documentId
- chunkText
- pageNumber optional
- title

Important:

Har chunk store karte waqt metadata zarur save karo.

Aaj ka end result:

Document searchable ho jaye vector DB me.

Day 18 — Query to embedding

Kya karna hai:

Jab user question puche:

- us question ko bhi embedding me convert karo

Example:

"budget kaha mention hua hai?"

Ye query embedding vector DB me search karega.

Aaj ka end result:

Question embedding ready ho.

Day 19 — Similarity search

Kya karna hai:

- top 3 ya top 5 relevant chunks laao
- vector similarity search use karo

Result:

Question ke closest chunks milenge

Example:

User asked: "Main conclusion kya hai?"

System returns:

- chunk 7
- chunk 11
- chunk 14

Aaj ka end result:

Relevant chunks successfully retrieve ho.

Day 20 — Final RAG answer generation

Ab final step:

Flow:

1. user question
2. query embedding
3. top chunks retrieve
4. prompt build karo
5. AI ko bolo:
"sirf provided context ke basis par answer do"

Prompt structure:

- system instruction
- retrieved chunks
- user question

Rule:

AI ko bolo hallucinate mat kare.

Agar answer context me nahi hai, bolo "document me ye info nahi mili".

Aaj ka end result:

User document se question puch ke answer le sake.

Day 21 — Chat UI

Kya karo:

- document detail page me chat section banao
- user question input
- answer cards
- loading state
- source chunks optional

Bonus:

- source snippets dikhao
- "view context" button do

Aaj ka end result:

Real AI document chat ready.

WEEK 4 — Automation features + polish

Day 22 — AI summary feature

Kya karna hai:

- selected document ka summary generate karo

Types of summary:

- short summary
- detailed summary
- bullet summary

UI me buttons:

- Generate summary
- Regenerate

Aaj ka end result:

Document summary working.

Day 23 — Key points extraction

Kya karna hai:

- AI se key takeaways nikalo
- important points numbered list me dikhao

Example:

- main objective
- main findings
- risks
- conclusion

Aaj ka end result:

Key insights feature ready.

Day 24 — Flashcard generator

Kya karna hai:

- document text se Q/A style flashcards banao

Output example:

Q: RAG kya hota hai?

A: Retrieval Augmented Generation...

UI:

- next/prev card
- flip card
- save flashcards

Aaj ka end result:

Flashcard feature ready.

Day 25 — Quiz generator

Kya karna hai:

- AI se MCQ quiz generate karo
- har question ke 4 options
- correct answer save karo
- score calculate karo

UI:

- Start Quiz
- Select answer
- Submit
- Show score

Aaj ka end result:

Quiz feature ready.

Day 26 — Chat history save

Chat model fields:

- userId
- documentId
- question
- answer
- sources
- createdAt

Kya karo:

- har chat save karo
- previous chats load karo

Aaj ka end result:

Conversation history ready.

Day 27 — Document management polish

Features:

- rename document
- delete document

- search documents
- filter by date
- status badges

Aaj ka end result:

Dashboard professional lage.

Day 28 — Better UX

Add:

- skeleton loaders
- toasts
- modals
- empty state
- failed processing retry
- mobile responsiveness

Aaj ka end result:

App polished lage.

Day 29 — Deployment

Frontend deploy:

- Vercel / Netlify

Backend deploy:

- Render / Railway

Database:

- MongoDB Atlas

File storage:

- Cloudinary / local temporary / AWS S3 later

Secrets:

- env variables properly set karo

Aaj ka end result:

Live project online.

Day 30 — Resume + portfolio + GitHub

GitHub pe:

- clean README
- project screenshots
- features
- tech stack
- architecture flow
- live link
- demo video

Portfolio me likho:

- AI Document Intelligence SaaS
- Built full-stack MERN application with RAG pipeline
- Implemented PDF parsing, chunking, embeddings, vector search, and AI-powered Q&A
- Added summary, flashcards, and quiz generation from uploaded documents
- Secured app with JWT auth and personalized document dashboards

Aaj ka end result:

Project job-ready ban jaye.

Exact build order in one line

Agar pura roadmap yaad nahi rakhna, bas ye order yaad rakho:

UI → Auth → Upload → PDF parse → Chunk → Embedding → Vector store → Similarity search → AI answer → Summary → Flashcards → Quiz → Deploy

Backend API routes structure

Ye structure follow kar sakte ho:

Auth

- POST `/api/auth/register`
- POST `/api/auth/login`
- GET `/api/auth/me`

Documents

- POST `/api/documents/upload`
- GET `/api/documents`
- GET `/api/documents/:id`
- DELETE `/api/documents/:id`
- PATCH `/api/documents/:id`

AI

- POST `/api/ai/ask/:documentId`
- POST `/api/ai/summary/:documentId`
- POST `/api/ai/flashcards/:documentId`
- POST `/api/ai/quiz/:documentId`

Chats

- GET `/api/chats/:documentId`

Database models simple version

User

- name
- email
- password

Document

- userId
- title
- fileUrl
- extractedText
- summary
- status

Chat

- userId
- documentId
- question
- answer
- sources

Flashcard

- userId
- documentId
- question
- answer

Quiz

- userId
- documentId
- questions[]

Frontend pages structure

- /
- /login
- /signup
- /dashboard
- /documents/:id
- /documents/:id/chat
- /documents/:id/summary
- /documents/:id/flashcards
- /documents/:id/quiz

Sabse important logic flow

Ye tumhare dimaag me permanently baith jana chahiye:

Upload flow

1. user PDF upload karta hai
2. backend PDF save karta hai
3. backend PDF text extract karta hai
4. text chunks me todta hai
5. embeddings generate karta hai
6. vector DB me save karta hai
7. document status completed karta hai

Ask question flow

1. user question bhejta hai
2. question embedding banta hai
3. vector DB top relevant chunks deta hai
4. AI ko chunks + question diya jata hai
5. AI answer banata hai
6. frontend answer show karta hai
7. optionally sources bhi show hote hain

Kaunse features sabse pehle banana hai

Agar time kam ho, pehle ye banao:

Must-have

- auth
- upload PDF
- parse PDF

- chunking
- embeddings
- vector search
- AI Q&A
- document dashboard

Good-to-have

- summary
- key points
- flashcards
- quiz
- chat history

Advanced

- multi-document chat
- citation highlighting
- OCR
- team workspace
- subscription billing
- admin panel

Internship ke liye sabse best version kaunsa hai

Tumhe sab kuchh perfect nahi banana.

Tum internship ke liye ye version banao:

Version 1

- login/signup
- dashboard
- upload PDF
- ask questions from PDF
- summary generation
- responsive UI
- deployed link

Bas ye sahi se ban gaya to bhi strong project hai.

Version 2

- flashcards
- quiz
- chat history

- better UI

Version 3

- multi-doc search
- subscriptions
- team workspace

Common mistakes jo nahi karna

1. Pehle hi advanced UI me mat faso

Pehle functionality.

2. Full PDF ek baar me AI ko mat bhejo

Chunking compulsory.

3. Metadata save kiye bina vector mat store karo

Baad me filtering mushkil ho jayegi.

4. Har jagah AI mat ghusa do

Useful jagah pe hi.

5. Har din feature add mat karo, testing bhi karo

Bina testing project tutega.

Tumhare liye best execution style

Tumhare liye best hoga:

Daily pattern

- 30 min: previous code samjho
- 2 hr: new feature build
- 45 min: bug fix
- 30 min: notes likho
- 15 min: GitHub push

Har din Git commit kaise karna

Commit messages:

- setup frontend and backend
- add jwt auth flow
- implement pdf upload api
- extract text from uploaded pdf
- add chunking utility

- integrate embeddings generation
- implement rag question answering
- add ai summary feature
- build flashcard generator
- deploy full stack app

Isse recruiter ko dikhega tumne structured kaam kiya hai.

Final project title options

Resume ke liye achhe names:

- DocMind AI
- InsightPDF
- AskDocs AI
- SmartDoc SaaS
- RAGDoc AI

Resume line ready-made

Built an AI-powered Document Intelligence SaaS using MERN, enabling users to upload PDFs, perform context-aware question answering through a RAG pipeline, and generate summaries, flashcards, and quizzes from document content.

Sabse pehle kal kya karna hai

Kal se start karna hai to exact order ye lo:

Kal ka task

1. project name decide karo
2. features list likho
3. frontend + backend folders banao
4. React app setup
5. Express server setup
6. MongoDB connect
7. GitHub repo create
8. first commit push

Bas. Itna karo.

Uske agle din

1. User model
2. Register route
3. Login route
4. JWT
5. Frontend login/signup pages

6. dashboard dummy UI

Uske agle din

1. Multer setup
2. PDF upload API
3. Upload UI
4. document model

Yehi sahi direction hai.

Agar chaho, next message me main tumhe iska **exact folder structure + database schema + phase 1 ka code order** bhi de deta hoon.